

Astria Backend Mars Image Tiler Service

Marsette Vona <marsette.a.vona@jpl.nasa.gov>

September 10, 2026

Contents

1	Introduction	3
2	Building	4
3	Command Line Interface Mode	4
4	AWS Lambda Mode	4
5	REST Service Mode	5
5.1	Non-Image Requests	5
5.2	Image Requests	5
5.2.1	Optional URL Parameters for Image Requests	6
5.2.2	Metadata JSON	8
5.2.3	Overlay Preferences	11
5.3	POST Requests	12
5.4	Finder API	12
6	REST Service Configuration	15

1 Introduction

The Mars Image Tiler (`tile_service`) Tomcat servlet generates **Deep Zoom** (DZI) products with PNG (default) or JPEG tiles for VIC, IMG, PNG, JPEG, and TIFF images including overlayable images with transparency. Several auxiliary products are also produced including a thumbnail and JSON metadata.

When converting a source VIC or IMG that is not an overlay type (e.g. EDR, FDR, RAS) the source data is stretched to 8 bit RGB. In most cases the default stretch is a histogram percent stretch with low=0 and high=0.5%. The default stretch is disabled for user uploads, quicklook/autolook/closerlook products, and EDL camera images.

When converting a source VIC or IMG that is an overlay type (e.g. XYZ, UVW, ARM) the source data is converted to an 8 bit RGB overlay image using `Tile_service` desktop library code.

When converting a source PNG or JPEG stretching is performed the same as for IMG or VIC. Overlay conversion is never performed for PNG or JPEG.

The tile service has no hard limit on the size of input images in VIC or IMG format, though there may be limits in practice e.g. due to resource constraints. Currently the tile service does have a limit of 2 gigapixels for input images in other formats including PNG, TIFF, and JPEG.

An alpha channel is added to the output by default if the output type is PNG and either overlay conversion was performed or the input was a CRISP (Co-Registration of Images Server Protocol) image, detected by checking if the filename starts with “warped-”. In each case a binary mask image is generated, and the output alpha channel has two values depending on whether a pixel is masked or not. For an overlay the mask corresponds to invalid and/or missing pixels in the source image. For a CRISP image, the mask corresponds to invalid, missing, and/or black pixels in the source image. In both cases the alpha value for masked pixels is zero (fully transparent). For an overlay, the default alpha value for unmasked pixels is 0.86 (translucent). For a CRISP image the default alpha value for unmasked pixels is 1 (fully opaque).

Unless it appears that Gamma (i.e. linear to sRGB) conversion has already been applied to the input, it will be applied while computing the tiling by default.

To avoid doubly computing and caching tilings the service will

- cache tilings computed for a .VIC (case insensitive) under the same path as for the corresponding .IMG
- return a cached tiling for the corresponding .IMG, if any, even when the original request was for a .VIC (note that in this case the requested .VIC must still exist and be readable by the client)

The service can run in three modes: command line interface (cli), as an AWS Lambda, and as a REST service. In a typical deployment the Lambda and REST

service share the same cache bucket and should be configured with the same environment variables, which define the default processing options. The Lambda pre-processes image products when they are created in S3. When a product is requested with default processing and it already exists in the cache bucket, the REST service can avoid significant processing. Non-default processing or requests that arrive before the Lambda has completed preprocessing are computed by the REST service and cached.

2 Building

1. Install latest [JDK 21](#). Make sure the environment variable `JAVA_HOME` is set properly, e.g. `$JAVA_HOME/bin/java` should be the java runtime executable. Also make sure that `javac` and `java` are added to `PATH`.
2. Install latest [Maven](#). Ensure the `mvn` executable is added to your `PATH`.
3. Configure Maven in `~/.m2/settings.xml` as necessary to access the dependency repositories.
4. `mvn package` should build `target/image_tiler.jar`.
5. `mvn -P war-FLAVOR package` should build `target/image_tiler.war`, where `FLAVOR` is one of `docker`, `localhost-fscache`, or `localhost-s3cache`.
6. All build files are created under the relative directory `target/`, which is git ignored. Recursively delete it to start fresh (though you may still have cached maven dependencies).

3 Command Line Interface Mode

The CLI can be used to batch convert images and manually process images on specific hardware. The input and output can be either the local filesystem or S3. Default conversion in CLI mode uses the same environment variables as the Lambda.

The CLI uses the `image_tiler.jar` build artifact. Run

```
java -jar image_tiler.jar -h
```

for a list of available options.

4 AWS Lambda Mode

The Lambda can be deployed to automatically convert images when they are added to an S3 bucket. It will also delete prior results when images are deleted from the bucket. Only default conversion is supported with the Lambda.

Since the Lambda may have both limited RAM and processing time, there may be input images which are too large for it to process. The Lambda can optionally be configured to detect such oversize inputs and forward them for processing to the REST service via an SQS queue.

Lambda deployments use the `image_tiler.jar` build artifact.

5 REST Service Mode

The REST service can convert images on demand, including with custom conversion parameters. It can also serve previously computed and cached results. It can read inputs and save cached results either on S3 or its local filesystem. It can also read inputs from any HTTP(S) server.

The REST service may consist of a single server or multiple servers under a load balancer. It may also be deployed behind an authentication proxy. REST service deployments use the `image_tiler.war` build artifact on Apache Tomcat.

There is no explicit authentication for files read from the local filesystem, but external access to the REST service can be gated behind an authentication proxy. When reading from S3 the server may either be configured to use a configured AWS profile or client credentials for the request passed on via an `X-AWS` header from an authentication proxy. See `aws_profile`, `aws_region`, `use_aws_header`, and `verify_client_credentials` below.

5.1 Non-Image Requests

A request like `https://SERVER/image_tiler/version` will return the deployed version.

A request like `https://SERVER/image_tiler/image_descriptions.xml` will return the `Tile_service` image descriptions XML file.

A request like `https://SERVER/image_tiler/openseadragon.min.js` will return a copy of the [OpenSeadragon](#) Deep Zoom Image viewer client library.

If the requested path is in the form of a cached tile image, that tile image is served.

5.2 Image Requests

A request like any of the following requests conversion of the image at the given location on S3, HTTP(S), or the server filesystem (latter when `image=PATH`):

- `https://SERVER/image_tiler?image=s3://BUCKET/KEY`
- `https://SERVER/image_tiler?image=http[s]://BUCKET.s3-REGION.amazonaws.com/KEY`
- `https://SERVER/image_tiler?image=http[s]://BUCKET.s3.REGION.amazonaws.com/KEY`
- `https://SERVER/image_tiler?image=http[s]://BUCKET.s3.amazonaws.com/KEY`
- `https://SERVER/image_tiler?image=http[s]://s3-REGION.amazonaws.com/BUCKET/KEY`
- `https://SERVER/image_tiler?image=http[s]://s3.amazonaws.com/BUCKET/KEY`
- `https://SERVER/image_tiler?image=http[s]://PATH`
- `https://SERVER/image_tiler?image=PATH`

If **REGION** is included and differs from the region of the AWS credentials (see below) the query will likely fail. `..` is disallowed in **KEY** and **PATH**.

If the requested conversion has not already been cached it is initiated and the call will block until the product has been created.

Image requests return the DZI XML including a **Url** attribute that is the base URL for DZI image tiles, which may be:

- this REST service itself if the conversion is still being computed or the cache is not on S3
- a mission venue S3 data proxy, if configured
- or the standard AWS https S3 proxy (e.g. `https://CACHE_BUCKET.s3-us-gov-west-1.amazonaws.com`)
- any service as e.g. `https://some.server.com/foo/bar/HASH/image_files/` configured with `tile_url`; **HASH** will be replaced with the image cache hash
- any service as `PROTOCOL://HOST:PORT/PATH/HASH/SUFFIX/` configured with `dzi_protocol_override`, `dzi_host_override`, `dzi_port_override`, `dzi_path_override`, and `dzi_suffix_override`; **HASH** will be replaced with the image cache hash.

5.2.1 Optional URL Parameters for Image Requests

The following optional URL parameters are only valid in combination with **image**:

- **viewer=true**: Returns a DZI viewer page instead of the DZI.
- **thumb=true**: Returns the thumbnail (always a PNG) instead of the DZI.
- **metadata=true**: Returns the [metadata JSON](#) instead of the DZI.
- **progress=true**: Returns progress of an ongoing DZI computation as a percent in the range 0.00 to 100.00 instead of the DZI, 100.00 if already computed, or HTTP 404 if not found.
- **error=true**: Returns error text of a failed DZI computation if any, or HTTP 404 if none.
- **force=true**: Force (re)computation of a tiling even if one already exists, appears to be in progress, appears to have failed, or if the image URL would have been filtered (except for security filters, which cannot be disabled). Ignored for auxiliary requests including **viewer=true**, **thumb=true**, **metadata=true**, **progress=true**, **error=true**.
- **maxWaitSec=NUMBER**: Override the default maximum wait time if non-negative. See `max_wait_time` below.
- **filter=false**: Override the server setting for `filter_image_urls`. With filtering disabled the REST service will still apply filtering related to security but will otherwise allow requests for specific images that would typically not be tiled automatically. Such tilings will be computed by the REST server on demand so this should be used sparingly.
- **forceCustom=true**: Consider the request to be a custom (non-default) processing even if it appears to have equivalent settings to what would

currently be considered default. Useful in situations where server updates change the default settings.

- **proxy=http|https|s3|data|tile|default:** Force the specified proxy type. **http** and **https** force using this REST server itself as the proxy. **s3** uses the standard AWS https S3 proxy `https://CACHE_BUCKET.s3.us-gov-west-1.amazonaws.com`, with AWS credentials passed as URL parameters (the server AWS credentials are always used to access the cache bucket, but if **verify_client_credentials** is set, the client request contained credentials, and **use_aws_header** is not **never**, it is first verified that the client can access the original image file). **data** uses the mission venue S3 data proxy `data_url`, if configured. **tile** uses `tile_url`, if configured. **default** automatically determines the proxy type.
- **rdr=TYPE:** override inferring the mission-specific image type from the source product ID. Must match an **image_type** id attribute in `tile_service_image_config.xml`. This is supported so that the REST service can be mission independent. The client may retrieve the image type e.g. from OCS where it may have been stored using mission specific ingestion.
- **format=png, format=jpg, format=jpeg:** Create image tiles in the requested format. Default is typically **png** but can be overridden in server config.
- **stretchLow=NUMBER, stretchHigh=NUMBER:** override the default stretch parameters. See **stretchType**.
- **stretchType=auto|percent|extrema|relative|none:** Default **auto** which typically implies **percent** but that can be overridden in the server config. For user uploads, autolook/quicklook/closerlook products, and EDL camera images **auto** implies **none**. For **stretchType=percent** **stretchLow** and **stretchHigh** are clamped to the range [0,100] (default 0 and 0.5) and interpreted as percents of the nonzero DN population to omit at the low and high end. For historical reasons this is done by clamping a corresponding number of histogram buckets at each end (the corresponding mode in desktop MarsViewer is called “histogram percent”). For **stretchType=extrema** **stretchLow** and **stretchHigh** are literal DN values and are required parameters. For **stretchType=relative** **stretchLow** and **stretchHigh** are clamped to the range [0,1] (default 1 and 1) and interpolate between the min/max actual DN in the image and the min/max representable DN for integer images (for float images **stretchType=relative** is always equivalent to **stretchType=extrema** with the min/max actual DN in the image).
- **overlayPreference=TYPE%3DNAME%3DVALUE:** If one or more of these (where the TYPE, NAME, and VALUE strings are also properly URL encoded) are provided they will be used as special processing options when creating the overlay. Default empty. See [below](#) for details.
- **tileSize=NUMBER:** Leaf tile size in pixels. Default is typically 254 but can be overridden in server config.
- **tileOverlap=NUMBER:** Tile overlap in pixels. Default is typically 1 but

can be overridden in server config.

- **thumbHeight=NUMBER**: Thumbnail height in pixels. Default is typically 160 but can be overridden in server config.
- **maskBlack=auto|true|false**: Include black in addition to any invalid or missing pixel value when creating the alpha mask. Auto means true iff input is a CRISP image. Default is typically auto but can be overridden in server config.
- **overlayAlpha=NUMBER**: Override the default unmasked overlay alpha. Default is typically 0.86 but can be overridden in server config.
- **unmaskedAlpha=NUMBER**: Override the unmasked alpha. Default is **overlayAlpha** for an overlay, else typically 1, but can be overridden in server config.
- **maskedAlpha=NUMBER**: Override the masked alpha. Default is typically 0 but can be overridden in server config.
- **gammaMode=MODE**: Override the default gamma mode. See **gamma_mode** below.

5.2.2 Metadata JSON

The metadata JSON contains the following entries:

- **rdr_url** (string): source image URL. Usually this is identical to the requested image URL. However to avoid double-caching the same tiling, if a .VIC or .IMG (case insensitive) URL is requested, the other one may be returned.
- **product_type** (string): source image product type, three letter code from product ID
- **rdr_type** (string): source image type, derived from **product_type**
- **tile_format** (string): tile format, typically **png**
- **tile_size** (number): leaf tile size, default 254
- **tile_overlap** (number): tile overlap, default 1
- **thumb_height** (number): thumbnail height, default 160 (width is computed to maintain image aspect ratio)
- **mask_black** (boolean): whether black is also considered a mask value in addition to any invalid or missing pixel values. Default is true for CRISP images. Note that invalid and missing pixel values are themselves commonly black.
- **unmasked_alpha** (number): alpha value for unmasked pixels, 0 is fully transparent and 1 is fully opaque
- **masked_alpha** (number): alpha value for masked pixels, 0 is fully transparent and 1 is fully opaque

- **gamma_mode** (string): setting of **gamma_mode** (see below) that was in effect when this image was processed. Also see **linear_to_srgb** and **srgb_to_linear** which record the actual conversion(s) applied as a result of **gamma_mode** and the image properties.
- **tile_header_gamma** (string): either **linear** or **srgb**. Tiles are typically written in the sRGB colorspace as is typical for PNG and JPG images used on the web. For certain settings of **gamma_mode** and/or certain kinds of input image, tiles may be written in linear colorspace instead of sRGB. Currently this is only implemented if the tile format is PNG. Also, it is currently common that tile image files may claim to be sRGB in their header metadata, but actually contain linear colorspace pixel data. **tile_header_gamma** should only be set to **linear** when the tile images are truly linear, both in their header metadata and their pixel data.
- **tile_data_gamma** (string): either **linear** or **srgb**. Expected gamma of tile image pixel data. Unfortunately, currently this may differ from the tile image header metadata. See **tile_header_gamma**.
- **tiler_version** (string): version of the image tiler that processed this product
- **processing_timestamp** (string): image tiler processing time as an ISO8601 timestamp
- **cache_hash** (string): hash code used to cache tiling results, typically used as a folder name in the cache bucket
- **custom** (boolean): whether the tiling parameters were treated as nominal/default or custom
- **rdr_size** (number): size of source image file in bytes
- **rdr_timestamp** (string): last modified time of source image as an ISO8601 timestamp
- **rdr_etag** (string): eTag (typically MD5) of source image
- **overlay_preferences** (JSON array of strings): custom overlay preferences typically in the format TYPE:NAME=VALUE.
- **is_overlay** (boolean): whether the data was processed as an overlay
- **width, height** (number): original full image size
- **original_is_float** (boolean): whether original image is floating point
- **original_bits** (number): bit depth of original image without overlay or stretch
- **original_bands** (number): number of bands in original image without overlay or stretch

- `overlay_bands` (number, only present if `is_overlay=true`): number of bands in overlay image
- `unstretched_min`, `unstretched_max` (array of number, only present if not `stretch_type=none`): min/max per-band values in the image after overlay conversion, if any, but before stretching
- `min_max_estimated` (boolean): true iff `unstretched_min` and `unstretched_max` were estimated from `histogram` rather than exhaustively computed
- `stretch_type` (string): one of `histogram_percent`, `extrema`, `relative`, or `none`
- `stretch_low_percent`, `stretch_high_percent` (number, only present if `stretch_type=histogram_percent`): parameters for `histogram_percent` stretch
- `stretch_low_relative`, `stretch_high_relative` (number, only present if `stretch_type=relative`): parameters for `relative` stretch
- `stretch_low`, `stretch_high` (number, only present if not `stretch_type=none`): raw stretch DN values
- `histogram_bins` (number, only present if not `stretch_type=none`): number of bins in histogram
- `histogram_low_inclusive` (number, only present if not `stretch_type=none`): lower limit of first histogram bin
- `histogram_high_exclusive` (number, only present if not `stretch_type=none`): upper limit of last histogram bin
- `histogram` (array of array of number, only present if not `stretch_type=none`): histogram of image before stretching. `original_bands` length array of `histogram_bins` length arrays.

`histogram[b][i]` counts pixel values v in band b in range $l + i * w \leq v < l + (i + 1) * w$

where $l = \text{histogram_low_inclusive}$

and $w = (\text{histogram_high_exclusive} - \text{histogram_low_exclusive}) / \text{histogram_bins}$.

- `linear_to_srgb` (boolean): whether a linear to sRGB (gamma) conversion was applied
- `srgb_to_linear` (boolean): whether a sRGB (gamma) to linear conversion was applied

5.2.3 Overlay Preferences

The format used when setting a content property is the following:

```
TYPE:propertyName=value
```

where **TYPE** is the type of the image content (XYZ, SEL), **propertyName** is the name of the property you are setting, and **value** is the overriding value of that property.

Only some primitives and arrays of primitives are legal values. Accepted primitives are: boolean (true, false); int (0,1); long (0L, 1L); double (0.0, .1); float (0.0f, .1f); and strings. Arrays and strings with spaces or special characters must be enclosed in quotes.

For example, to set the property foo for image type IMG to an array of doubles, a legal format would be:

```
IMG:foo="[0.0,1.1,.2]"
```

There is no type widening or narrowing. Therefore, arrays cannot mix types (i.e. [1,1.2] is not allowed).

- I. Stretchable Images
 - A. Generic Images
 - Types: EDR, EFF, FFL, etc
 - Properties:
 - dataRangeMin - double
 - dataRangeMax - double
 - B. Formatted Images
 - Types: Disparity, IEP, IFF, Rough, Normal, Solar Engery, Slope, Slope Heading, Slope Magnitue
 - Properties:
 - dataRangeMin - double
 - dataRangeMax - double
- II. Contour Images
 - A. Contour Stretch
 - Types: XYZ, Range, XXX, YYY, ZZZ
 - Properties:
 - contourWidth - double[]
 - contourInterval - double[]
 - contourOffset - double[]
 - majorContourFactor - int[]
 - invalidValue - double[]
 - axisDn - int
 - majorContourDn - int
 - minorContourDn - int
 - backgroundDn - int

```

        invalidDn - int
    B. Contour with Stretchable Overlay
        Types: XXX, YYY, ZZZ
        Properties:
            dataRangeMin_overlay - double
            dataRangeMax_overlay - double
III. Other Images
    A. Mask Images
        Types: Mask
        Properties: None
    B. Reachability Images
        Types: IDL Reachability
        Properties:
            redBand - int
            greenBand - int
            blueBand - int

```

5.3 POST Requests

POST requests are also accepted with `application/json` bodies. The JSON content must be an object that includes the key `rdr_url`. It may also optionally include `force`, `max_wait_sec`, `rdr_type`, `tile_format`, `gamma_mode`, `tile_size`, `tile_overlap`, `thumb_height`, `percent_stretch`, `mask_black`, `stretch_low`, `stretch_high`, `overlay_alpha`, `unmasked_alpha`, `masked_alpha`, and `overlay_preferences`.

5.4 Finder API

A finder API can optionally be exposed (see `enable_finder` below) to support missions that do not have another service to provide metadata about available image products. Currently the finder API is only implemented for files in a local filesystem hierarchy under `fs_input_dir`, which must also be defined to enable the finder API. The finder API is also currently only implemented for the M2020 and CADRE missions.

There is no explicit authentication for access to the local filesystem with the finder API, but access to the REST service can be gated behind an authentication proxy.

When the finder API is enabled the following additional GET requests are supported.

A request like `https://SERVER/image_tiler/finder/solrange` will return a JSON object

```
{ "start": M, "end": N }
```

where M and N are either -1, indicating no image data is available, or non-negative integers giving the minimum and maximum inclusive sol range for which image data is available. Note: depending on the setting of `finder_max_sol_range_age` the range of available sols may be cached on the server and only recomputed periodically.

A request like `https://SERVER/image_tiler/finder/edrs?sol=SOL` will return a JSON array

```
[ { "edr": "PATH", "metadata": { ... } }, ... ]
```

with one entry per available image EDR (engineering data record) for the given sol. The array will be empty if no image EDRs are available for that sol. The metadata dictionary includes the following entries, which are all based on the pathname of the EDR:

- `"finder_type": <TYPE>` where `<TYPE>` is one of "Single Frame", "Mosaic", or "Tile"
- `"instrument_id": <ID>` where `<ID>` is a mission-specific instrument ID, e.g. "FL", "FR", "FA", "NL", "NR", "NA", "ZL", "ZR", "ZA", etc
- `"instrument": <INST>` where `<INST>` is a mission-specific instrument name, e.g. "fcamAL", "fcamAR", "fcamAS", "ncamL", "ncamR", "ncamS", "zcamL", "zcamR", "zcamS", etc
- `"instrument_category": <CAT>` where `<CAT>` is a mission-specific instrument category, e.g. "fcam", "ncam", "zcam", etc
- `"product_type": <PROD>` where `<PROD>` is a mission-specific product type, e.g. "RAS", "XYZ", etc
- `"group_id": <GROUP>` where `<GROUP>` is a mission-specific identifier of the image group, which may include different stereo eyes, product types, size types, etc; e.g. "011406770709389930110000ncam_OA0"
- `"overlay_id": <ID>` where `<ID>` is a mission-specific identifier of the overlay group, which typically partitions the image group into overlayable subsets e.g. by separating stereo eye, geometry type, size type, etc
- `"is_source_product": true|false` true iff the product type is suitable for use as an underlay, e.g. RAS, RAD, etc
- `"overlayable": true|false` true iff the product type is suitable for use as an overlay, e.g. XYZ, UVW, etc
- `"eye_type": <EYE>` where `<EYE>` is one of "Left Eye", "Right Eye", "Mono Eye", "Stereo Pairs", "Anaglyph", "Colorglyph", "Color Stereo", "Mixed"
- `"size_type": <SIZE>` where `<SIZE>` is one of "Full", "Thumbnail"
- `"projection": <PROJ>` where `<PROJ>` is one of "Cylindrical", "Perspective", "Cylindrical Perspective", "Polar", "Orthographic", "Orthorectified", "Vertical", "Sinusoidal"

- "geometry": <GEOM> where <GEOM> is one of "Raw", "Nominal", "Actual", "Trapezoid"
- "stage": <STAGE> where <STAGE> is one of "Primary", "Secondary", "Special"
- "version": <VER> where <VER> is a mission-specific version; e.g. "01", "B3", etc
- "sclk": <SCLK> where <SCLK> is a mission-specific SCLK string or NULL if not implemented; e.g. "0677070938"
- "sol": <SOL> where <SOL> is an integer sol number, or day-of-year in the range 1-365 iff sol_doy is true, or -1 if not implemented
- "sol_doy": true|false indicating whether sol is a day-of-year
- "year": <YEAR> where <YEAR> is an integer giving the calendar year of sol
- "image_type": <TYPE> where <TYPE> is a mission specific image type, e.g. "edr", "color_rgb", "xyz", etc
- "description": <DESC> where <DESC> is a description of the product type
- "supplemental_description": <DESC> where <DESC> is a supplemental description of the product type.

With a few exceptions, e.g. "finder_type" (which replaces "PRODUCT_OCS_TYPE"), "instrument", and "sclk" (which is called "time2" in OCS), these are a subset of the data available in the mission OCS (operational cloud store).

A request like https://SERVER/image_tiler/finder/edr?image=PATH will return a JSON object

```
{ "edr": "PATH", "metadata": { ... } }
```

with the same metadata as above. The server will return HTTP 404 if the given EDR is not found.

A request like https://SERVER/image_tiler/finder/rdrs?edr=PATH will return a JSON array

```
[ { "rdr": "PATH", "metadata": { ... } }, ... ]
```

with one entry per available image RDR (reduced data record) associated with the given EDR. The server will return HTTP 404 if the given EDR is not found. The array will be empty if the EDR is found but no image RDRs are available for it. The metadata dictionary includes the same entries as above.

A request like https://SERVER/image_tiler/finder/rdr?image=PATH will return a JSON object

```
{ "rdr": "PATH", "metadata": { ... } }
```

with the same metadata as above. The server will return HTTP 404 if the given RDR is not found.

A request like `https://SERVER/image_tiler/finder/label?image=PATH` will return a JSON object

```
{ "system": { ... }, "properties": { ... }, "tasks": [ ... ] }
```

if the given path ends in `.VIC` or `.IMG` (case insensitive) and contains either plain or PDS/ODL wrapped **VICAR** data. The server will return HTTP 404 if the given path is not found. Each value in the returned label data is either a JSON string, number, array of strings, or array of numbers corresponding to the label value. The **system** dictionary contains all the VICAR system labels that were present in the file except `LBLSIZE` and `EOL`. See the VICAR documentation for descriptions of the possible system labels, which ones are mandatory, and the default values for the others. The **properties** dictionary contains one entry for each property set in the VICAR file (if there are none then the **properties** dictionary will be present but empty). The name of each entry in the **properties** dictionary is the name of the corresponding property set, and value of the entry is another dictionary containing the VICAR labels for that property. The **tasks** array contains one entry for each task in the order they were given in the VICAR file (if there are none then the **tasks** array will be present but empty). Each entry in the **tasks** array is a dictionary containing the VICAR labels for that task, including an entry like `"TASK": "<name>"`.

6 REST Service Configuration

The following servlet initialization parameters may be specified in `web.xml` for the REST service. Uppercase versions of the same values are also recognized as environment variables which are used by the CLI, the Lambda, and in Docker deployments of the REST service. Several variables are different for the Lambda as noted below.

Note on default values: the defaults described here refer to the defaults in the code. They can be overridden on a per-venue basis.

- **debug_tiler**: Defaults to `false`. Set to `true` to enable extra logging.
- **format_json**: Defaults to `true`. Set to `false` to disable JSON formatting.
- **mission**: one of `MSL`, `M20`, `M2020`, or `CADRE`. Defaults to `M20`. Used to determine the format for parsing input filenames as mission product IDs when `rdr=TYPE` is absent.
- **enable_s3**: Must be `true`, `false`, or `auto`. Defaults to `true`. Enables reading inputs from and saving cached results to S3.
- **enable_http**: Must be `true`, `false`, or `auto`. Defaults to `true`. Enables reading inputs from `http[s]` URLs.
- **rdr_url_whitelist_patterns**: Defaults to empty, meaning allow all. Comma separated list of URL regex to whitelist RDR URLs. If this

is empty no whitelist is applied, but the `enable_s3` and `enable_http` settings are still respected. This whitelist is for security and cannot be disabled with the `force=true` or `filter=false` request options.

- `enable_sqs`: Must be `true`, `false`, or `auto`. Defaults to `false`. Enables the Lambda or REST service to use SQS features (not counting the normal input and fail queue for the Lambda). Ignored for CLI.
- `enable_sqs_heartbeat`: defaults to `false`. Only applies to Lambda. Enables SQS message visibility timeout heartbeating in the Lambda. This will keep a message invisible for approximately as long as the Lambda is working on it, but not too much longer. Requires `sqs:GetQueueUrl` and `sqs:ChangeMessageVisibility` permissions.
- `sqs_retries`: Defaults to 3. Max number of retries to process a DZI for request received by SQS. Only applies to REST service.
- `data_url`: optional mission S3 data proxy
- `tile_url`: optional tile service URL, e.g. `https://some.server.com/foo/bar/HASH/image_files/.HASH/` will be replaced with the image cache hash.
- `dzi_protocol_override`: If not null, empty, or `auto`, this overrides the protocol in the URL in DZI xml files (except when using the `data` proxy mode). Examples: “http”, “https”.
- `dzi_host_override`: If not null, empty, or `auto`, this overrides the host-name in the URL in DZI xml files (except when using the `data` proxy mode). Example: “foo.bar.com”.
- `dzi_port_override`: If not null, empty, or `auto`, this overrides the port number in the URL in DZI xml files (except when using the `data` proxy mode). Use -1 for no explicit port, -2 or lower to use same port as request. Default -2.
- `dzi_path_override`: If not null, empty, or `auto`, this overrides the path in the URL in DZI xml files (except when using the `data` proxy mode). Empty for no path, otherwise this should start with `/` and not end with `/`.
- `dzi_suffix_override`: If not null, empty, or `auto`, this overrides the path suffix in the URL in DZI xml files (except when using the `data` proxy mode). Empty for no suffix, otherwise this should start with `/` and not end with `/`.
- `redirect_cached`: Must be `true`, `false`, or `auto`. Defaults to `true`. Applies only to REST service. Instead of proxying requests for already cached files (image tiles and metadata files), redirect the request. The DZI xml files are always served by the REST service, but the embedded URL to the tiles is modified depending on the setting of `redirect_cached`, `data_url`, `tile_url`, the cache type (S3 or filesystem), the `proxy` request parameter, `serve_tiles_from_tasks`, and whether the image is still being processed.
- `allow_multihop_redirect`: Must be `true`, `false`, or `auto`. Defaults to `false`. Allow the server to respond to requests for image tiles or metadata files with a data proxy URL (using `data_url`) that will in turn (typically) redirect to a presigned https S3 URL. Such multihop redirects may cause CORS errors. When `false` such URLs will instead be formulated as direct

presigned https S3 URLs using the server credentials to access the cache bucket.

- **fs_input_dir**: optional content root for filesystem input images. Applies only to REST service. Local filesystem access is disabled if this is null or empty.
- **enable_finder**: Defaults to false. Enable the [finder API](#). Requires **fs_input_dir** to also be set. Currently only implemented for M2020 and CADRE.
- **max_sol_range_age**: Defaults to 300 (5 minutes). Maximum time between recomputing available sols in [finder API](#). Non-positive forces recompute for each **solrange** query.
- **cache_type**: must be **s3** or **fs**. Empty, null, or **auto** defaults to **s3**.
- **s3_cache_bucket_name**: name of cache bucket, required if **cache_type=s3**.
- **s3_cache_loc**: cache folder in S3 bucket, optional.
- **fs_cache_loc**: cache folder for filesystem cache, optional.
- **aws_profile**: Defaults to **null**. AWS credentials profile name. Use **null** for the [default credential provider chain](#).
- **aws_region**: Defaults to **us-gov-west-1**. AWS region name. Use **null** for to use the [default region provider chain](#).
- **use_aws_header** (environment variable **TILER_USE_AWS_HEADER**): one of **always**, **never**, or **prefer**. Defaults to **prefer**. Whether to use the X-AWS request header for credentials to read input images on S3. (IAM role credentials are always used to access the cache bucket on S3.) In **prefer** mode the X-AWS header is used if present, otherwise **aws_profile** in **aws_region** is used. In **always** mode the X-AWS header is always used and the request fails if it's absent. In **never** mode the X-AWS header is ignored and **aws_profile** and **aws_region** are always used. Applies only to REST service.
- **cache_s3_credentials**: Defaults to 64. When using X-AWS request headers for S3 credentials, cache up to this many credentials for potential near term re-use. Only applies to REST service.
- **max_s3_credentials_age**: Defaults to 900 (15 minutes). When using X-AWS request headers for S3 credentials, cache credentials for potential near term re-use up to this many seconds from when they were created. Only applies to REST service.
- **verify_client_credentials**: Must be **true**, **false**, or **auto**. Defaults to **true**. When using client S3 credentials but actually accessing cache bucket data with server credentials, verify that the client credentials have read access to the original image when serving a cached tile image.
- **tile_format**: Defaults to **png**. DZI tile image format.
- **tile_size**: Defaults to 254. DZI leaf tile image size. Set non-positive to use default.
- **tile_overlap**: Defaults to 1. DZI tile image overlap. Set negative to use default.
- **thumb_height**: Defaults to 160. DZI thumb image height. Set non-positive to use default. Thumbnails will generally be this tall. The width of the

thumbnail will be proportionate to match input image aspect ratio. The thumbnail may be shorter in cases where the input image was smaller than the requested thumb height, or (typically for wide aspect ratio input images) if downscaling the input image by the smallest power of two that makes it fit within a 4x4 tile grid results in a smaller vertical dimension than the requested thumb height.

- **dzi_parallelism**: Defaults to number of available CPU cores. Number of threads to use when computing a DZI. Set negative to use all available CPU cores.
- **read_chunk**: Optional k/m/g/% suffix. Defaults to 0 for lambda and REST service, 25% for CLI. Clamped to 2g max. Interpreted as percent of task memory budget when given as a percent. Interpreted as bytes when greater than one. Interpreted as fraction of decompressed source image when given in the range 0-1. Positive values increase the size of the chunks copied from the source image when building the DZI. Zero means that each leaf tile of the DZI will be read individually from the source image. 0.25 means that the original image will be read in four chunks corresponding to the four tiles below the root of the DZI image pyramid. For most image formats reading fewer but larger chunks can increase performance, but at the expense of increased memory usage. Note that there is usually a level of indirection between this and actual reads of the source image file, which will often be different depending on the format and image size. For example, large IMG/VIC are typically actually read in chunks of the JAI tile size. Smaller images of all types (below **large_image_threshold**) will generally be read in whole. Larger non-IMG/VIC may be read in windows, see **window_chunk** and **tile_window_large_non_pds**. Disabled automatically whenever the source image does not exceed **large_image_threshold** or when the read chunk would be smaller than the DZI leaf tile size. If the subimage of the source image corresponding to the read chunk would be 2 gigapixels or larger (i.e. would not fit in a Java byte array) then **read_chunk** will be automatically reduced.
- **tile_window_chunk**: similar to **read_chunk**. Defaults to 25%. Clamped to 2g max. When loading an image using tile windows (see **tile_window_large_non_pds**) then this determines the quadtree level of the reads. Note that setting this with a % suffix means it's relative to the task memory budget, but setting it in the range 0-1 means it's relative to the input image size. The effective window chunk as a fraction of the input image size will always be computed from this as the size of the nearest (smaller) level in the DZI tile quadtree. If the effective window chunk is 1 then tile window loading will be disabled and the full image will be read. If the effective window chunk is less than **min_tile_window_chunk** the image will be ignored.
- **min_tile_window_chunk**: Range 0-1. Defaults to 0. When loading an image using tile windows (see **tile_window_large_non_pds**), if the actual effective window chunk as a fraction of the image is less than this threshold, abort. Smaller tile window fractions mean the source image will be read

more times. Setting this threshold to 1 effectively disables loading images using tile windows. Lambda deployments may set this to e.g. 0.25 or 1 to reject images which would need to be read more than e.g. 4x or 1x. That can be desirable for the lambda if `large_image_threshold` is also set relatively high. Images larger than a hefty `large_image_threshold` that would require tile window reads (typically png/jpg/tiff but not vic/img) will often fail in a lambda anyway due to the processing time limit.

- `large_file_threshold`: Defaults to 128m, optional k/m/g/% suffix. No limit if negative. Interpreted as a percent of task memory budget when given as a percent. If the image file size in bytes exceeds this threshold then the input file will be considered large, possibly enabling alternate algorithms or more spew for slow operations.
- `large_image_threshold`: Optional k/m/g/% suffix. No limit if negative (no images considered large). Interpreted as a percent of task memory budget when given as a percent. Defaults to 128m for lambda and REST service, 50% for CLI. Clamped to 2g if positive. If the decompressed image size in bytes does not exceed this threshold and the image will fit in the available memory budget (or no limit) and then the image will be loaded in full. Disabled for IMG/VIC unless `full_load_small_pds=true`
- `full_load_small_pds`: Default false. See `large_image_threshold`.
- `limit_pds_height`: Default false. Clamp the maximum rows when loading IMG/VIC so that the total image size in *pixels* is less than 2G.
- `limit_non_pds_height`: Default false. Clamp the maximum rows when loading non-IMG/VIC so that the total image size in *bytes* is less than 2G.
- `tile_window_large_non_pds`: Default true. Load large non-IMG/VIC in quadtree chunks determined by `tile_window_chunk`. The input file will be re-parsed for each load.
- `max_histogram_bins`: default 4096. When computing a histogram for percent stretch, use at most this many bins. Fewer bins will be used for integer images where the number of significant bits results in fewer possible distinct values than the maximum number of bins.
- `histogram_mode`: one of `jai`, `reverseRaster`, `reverseQuadtree`. Default `reverseQuadtree`. For large images it can be more efficient to build the histogram in reverse access order of the DZI to increase cache hits.
- `never_estimate_extrema_from_histogram`: default false. Never estimate image extrema from histogram. Overrides `always_estimate_extrema_from_histogram`.
- `always_estimate_extrema_from_histogram`: default true. Always estimate image extrema from histogram even when random reads are not expensive. Applies only to integer images.
- `mem_cache_tiles`: Defaults to false. Serve DZI tiles from memory cache while the DZI is still being computed. Requires sufficient memory for entire tileset. Tiles are served from persistent storage (filesystem or S3) when the DZI computation is finished or this setting is false. Ignored (always false) for CLI and Lambda, where this feature would not be useful anyway. Because this can consume significant and variable memory per task, not recommended for use in the service either.

- **memory_budget**: optional, defaults to auto, memory budget for all DZI computation tasks, bytes with optional k/m/g suffix (kibi/mebib/gibi bytes), or “auto”, or numeric percent (0 to 100) with suffix %. Auto is equivalent to a default percent. Percent is relative to the maximum allowed size for the Java heap at runtime. Negative uses current free heap space. Zero disables LRU memory cache.
- **disk_budget**: optional, defaults to auto, disk budget for all DZI computation tasks, bytes with optional k/m/g suffix (kibi/mebib/gibi bytes), or “auto”, or numeric percent (0 to 100) with suffix %. Auto is equivalent to a default percent. Percent is relative to **lru_disk_cache_dir** free space at server start. Ignored (always 0) for the Lambda. Negative uses current free disk space. Zero disables LRU disk cache.
- **jai_cache**: optional, defaults to auto, JAI tile memory cache capacity, bytes with optional k/m/g suffix (kibi/mebib/gibi bytes), or “auto”, or numeric percent (0 to 100) with suffix %. There is one single JAI tile cache for the whole server. Auto is equivalent to a default percent. Percent is relative to **memory_budget**. Negative is equivalent to auto. Zero disables JAI tile cache.
- **pds_tile_width**, **pds_tile_height**: optional, defaults to -1, tile width/height for IMG/VIC, non-positive to auto compute. These only apply to IMG/VIC input images that are larger than **large_file_threshold** or **large_image_threshold**.
- **lru_page_bytes**: optional, defaults to 16m, LRU cache page size, bytes with optional k/m/g suffix (kibi/mebib/gibi bytes). Zero disables LRU memory and disk cache. Negative is equivalent to default.
- **lru_mem_cache_pages**: optional, defaults to auto, LRU memory cache size in pages, or “auto”, or numeric percent (0 to 100) with suffix %. This limit applies separately to each input image that is processed by the server. E.g. if 5 input images are concurrently being processed then up to 5 * **lru_mem_cache_pages** pages can be allocated. Auto is equivalent to a default percent. Percent is interpreted relative to **memory_budget** and **lru_page_bytes** so that each processed image may use up to the given percentage of memory for its LRU cache. When a task is launched its actual memory cache page limit is set to the minimum of **lru_mem_cache_pages** and the minimum number of pages that would be required to cache the entire input image. Ignored (always 100%) for the Lambda. Negative is equivalent to auto. Zero disables LRU mem cache.
- **lru_disk_cache_pages**: optional, defaults to auto, LRU disk cache size in pages, or “auto”, or numeric percent (0 to 100) with suffix %. This limit applies separately to each input image that is processed by the server. E.g. if 5 input images are concurrently being processed then up to 5 * **lru_disk_cache_pages** pages can be allocated. Auto is equivalent to a default percent. Percent is interpreted relative to **disk_budget** and **lru_page_bytes** so that each processed image may use up to the given percentage of disk for its LRU cache. When a task is launched its actual disk cache page limit is set to the minimum of **lru_disk_cache_pages** and

the minimum number of pages that would be required to cache the entire input image. Ignored (always 0) for the Lambda. Negative is equivalent to auto. Zero disables LRU mem cache.

- **lru_disk_cache_dir**: optional, defaults to auto, LRU disk cache directory, “none” disables. Each task that uses LRU disk cache makes a separate temporary unique subdirectory. “auto” uses OS temp dir.
- **ignore_subdirs**: Defaults to empty. Comma separated list of subdirectories to exclude from processing, case sensitive, e.g. `ids-pipeline,browse,orbital`. If any path segment of a request URL matches any ignored subdir the request will fail.
- **ignore_exts**: Defaults to empty. Comma separated list of filename extensions to exclude from processing, case insensitive, e.g. `png,jpg`.
- **ignore_exceptions**: Defaults to empty. Comma separated list of URL regex to specifically include in processing, overriding **ignore_exts**, case insensitive. Note that quicklook/autolook/closerlook, user uploads, and CRISP products have their own whitelists.
- **quicklook_patterns**: Defaults to `(?i).*/[AQC][^/]+SOL[^/]+[.] (PNG|JPG|JPEG|TIF|TIFF|IMG)$`. Comma separated list of URL regex to whitelist quicklook/autolook/closerlook products.
- **user_upload_patterns**: Defaults to `(?i)s3://[^/]*user-upload.*`. Comma separated list of URL regex to whitelist user uploads.
- **crisp_patterns**: Defaults to `.*warped-[^/]*`. Comma separated list of URL regex to whitelist and treat as CRISP overlay.
- **crisp_suffixes**: Defaults to `LUJ00,J00,00`. Comma separated list of alternate suffixes to use for short product IDs parseed out of CRISP filenames.
- **fifteen_bit_patterns**: Defaults to `R[AZ][DY],[CDZM]NR,[CDZM][SPW]D`. Comma separated list of regex for product type three letter codes to assume are 15 bit for 16 bit integer images when **SAMPLE_BIT_MASK** header is missing.
- **max_rdr_bytes**: Defaults to 0, optional k/m/g/% suffix. No limit if non-positive. Interpreted as percent of task memory budget when given as a percent. If positive (must be at least 1024) input images (compressed filesize in bytes on disk) bigger than this are ignored. Typically this is unlimited for the CLI and the REST service, where processing should always succeed for any input image size, though it will be significantly slower for images that don’t fit in cache. Typically this is set at some fraction of the provisioned memory for the Lambda because the Lambda is also strongly time and disk limited.
- **max_image_bytes**: Defaults to 0, optional k/m/g/% suffix. No limit if non-positive. Interpreted as percent of task memory budget when given as a percent. If positive (must be at least 1024) input images (decompressed bytes in memory) bigger than this are ignored. Typically this is unlimited for the CLI and the REST service, where processing should always succeed for any input image size, though it will be significantly slower for images that don’t fit in cache. Typically this is set at some fraction of the provisioned

memory for the Lambda because the Lambda is also strongly time and disk limited.

- **lambda_max_rdr_bytes**: similar to **max_rdr_bytes**. If set, this takes precedence over **max_rdr_bytes** for the Lambda. For the REST service, both values may be set: **max_rdr_bytes** controls the maximum size image that the REST service will itself attempt to compute, and **lambda_max_rdr_bytes** defines the maximum size image that the REST service will assume the Lambda should compute (see **lambda_wait_time**). For the REST service **lambda_max_rdr_bytes** cannot be specified as a percent.
- **lambda_max_image_bytes**: similar to **max_image_bytes**. If set, this takes precedence over **max_image_bytes** for the Lambda. For the REST service, both values may be set: **max_image_bytes** controls the maximum size image that the REST service will itself attempt to compute, and **lambda_max_image_bytes** defines the maximum size image that the REST service will assume the Lambda should compute (see **lambda_wait_time**). For the REST service **lambda_max_image_bytes** cannot be specified as a percent.
- **tile_service_queue_name**: Default null. Only applies to Lambda and REST service. If nonnull and nonempty and **enable_sqs** then the Lambda will forward requests for unhandled products (typically those larger than **lambda_max_rdr_bytes**) to this queue. The REST service listens on this queue. The messages must have JSON bodies in the same format as POST requests.
- **fail_queue_name**: Default null. Only applies to REST service. If nonnull and nonempty then the REST service will log failed processing here. The messages will have JSON bodies in the same format as POST requests.
- **tiler_max_concurrent_requests**: Defaults to -1 (unlimited). If positive then limit the number of concurrent requests. Additional load will result in failed requests. Applies only to REST service.
- **max_concurrent_tasks**: Defaults to -1 (unlimited). Integer, “auto”, or numeric percent (0 to 100) with suffix %. If positive then limit the number of concurrent DZI tasks. Additional load will result in failed requests. Auto is equivalent to a default percent. Percent is interpreted as $\text{max_concurrent_tasks} * (\text{memory_budget} - \text{jai_cache}) / (\text{task_overhead} + \text{lru_mem_cache_pages} * \text{lru_page_bytes})$. This is an upper limit; the server will also account actual task memory and disk usage and limit running tasks to **memory_budget** and **disk_budget**. Applies only to REST service.
- **task_pool_wait_time**: Time in seconds or h/m/s suffix, defaults to 30s. Applies only to REST service. If positive then wait before failing a request that will require computing a new DZI when **max_concurrent_tasks** are already running or insufficient memory or disk is available because of other running tasks. Zero disables waiting, negative uses default.
- **lambda_wait_time**: Time in seconds or h/m/s suffix, defaults to 0 (no wait). Applies only to REST service. If positive then wait before failing a

request for a product with nominal processing that is not already available in cache. Zero disables waiting, negative uses default.

- **max_zombie_time**: Time in seconds or h/m/s suffix, defaults to 1h. If a product has partially been computed in cache but has not updated its progress or PID in this amount of time then recompute it when it is next requested. Zero disables zombie check, negative uses default.
- **max_wait_time**: Time in seconds or h/m/s suffix, defaults to 30s. If positive, limits the wait when a product (e.g. DZI, thumbnail, tile, metadata, error) is served from a running task. Zero disables wait, negative uses default.
- **task_interlock_time**: Time in seconds or h/m/s suffix, defaults to 10s. If positive, wait this long before starting a new task to ensure that no other tasks are starting for the same product. Zero disables wait, negative uses default.
- **max_interlock_wait_time**: Time in seconds or h/m/s suffix, defaults to 2m. If positive, wait this long for other tasks to abort before starting a new task. Zero disables wait, negative uses default.
- **serve_tiles_from_tasks**: Must be **true**, **false**, or **auto**. Defaults to **true**. Serve tile images from a running task if not available in cache. If combined with a significant **max_tile_wait_time** this can lead to the some browsers (cough, Chrome) exhausting their socket pool while multiple connections are held open waiting for tiles to be computed. Applies only to REST service. It can be valuable to enable this when the frontend does not retry failed tile requests (which is typically the case) so that tiles requested before they have finished processing may still load after a short wait (but if the wait becomes too long they will still fail until the whole tileset is reloaded).
- **filter_image_urls**: Defaults to **false**. Apply same filtering policy to input image URLs as Lambda. Applies only to REST service. Can be overridden with **filter** URL parameter.
- **check_etag_value**: Must be **true**, **false**, or **auto**. Defaults to **true**. Validate eTag value in addition to timestamp to detect stale cache entries. Note that eTag value may differ even for the same source data depending on whether the data was processed from a local file (e.g. manual processing with CLI using pre-downloaded data) or directly from S3. Applies only to REST service.
- **percent_stretch**: Defaults to **true**. Whether to default to percent vs extrema stretch.
- **stretch_low**: Defaults to 0.5. Percent stretch low value in range 0-100, or “auto” to use default. Default for extrema stretch is always min band value in the image.
- **stretch_high**: Defaults to 0.5. Percent stretch high value in range 0-100, or “auto” to use default. Default for extrema stretch is always max band value in the image.
- **mask_black**: Defaults to **auto**. Must be **true**, **false**, **auto**, empty, or absent. Empty and absent are equivalent to **auto**. Whether to always

include black pixels in the alpha mask, potentially in addition to any other invalid or missing pixel values specified in the image header.

- **overlay_alpha**: Defaults to 0.86. Must be in the range 0 (fully transparent) to 1 (fully opaque) inclusive. Alpha value for unmasked overlay pixels.
- **unmasked_alpha**: Defaults to `auto`. Must be in the range 0 (fully transparent) to 1 (fully opaque) inclusive, or `auto`. Empty, absent, and out of range treated same as `auto`. Alpha value for unmasked pixels. Auto means use **overlay_alpha** for overlay, else 1.
- **masked_alpha**: Defaults to 0 for overlay and CRISP products, 1 otherwise. Must be in the range 0 (fully transparent) to 1 (fully opaque) inclusive. Alpha value for masked pixels.
- **gamma_mode=none|passthrough|convert|lineartosrgb|srgbtolinear**: Apply or unapply gamma (linear to sRGB) as a last step while making the DZI. Default is `convert`. `none` performs no explicit data conversion (though implicit conversions can and often will be applied based on the input image metadata) and always emits sRGB tile images. `convert` attempts to detect whether an explicit conversion to sRGB is needed. `passthrough` also performs no explicit data conversion, but attempts to detect whether the output tile images should be in the linear RGB colorspace instead of sRGB. Actual linear colorspace output tile image files may only work when **tile_format** is `png`.
- **preapplied_gamma_patterns**: Defaults to `[C,R].G`. Comma separated list of regex for product type three letter codes to assume preapplied gamma of 1/2.2 when `ENCODED_DISPLAY_GAMMA` header is missing. Only applies to IMG/VIC.
- **preapplied_8bit_gamma_patterns**: Defaults to `ECM,ECV`. Comma separated list of regex for product type three letter codes to force preapplied gamma of 1/2.2 when 8 bit. Only applies to IMG/VIC.
- **preapplied_inst_gamma_patterns**: Defaults to empty. Comma separated list of regex for instrument two letter codes to force preapplied gamma of 1/2.2. Only applies to IMG/VIC. Set to e.g. `E.,H.,PC` for EDLCam, Helicopter, and Pixa-MCC.